MASTER PLAN: Can LLM Critical Periods Be Reopened?

A Complete Step-by-Step Research Walkthrough (~12-13 weeks)

THE RESEARCH QUESTION

Children have "critical periods" — windows in early development where missing certain experience (like language exposure) causes effects that don't fully reverse later, no matter how much exposure comes afterward. Deep neural networks show an analogous effect (Achille et al. 2019).

Your study asks three linked questions about LLMs specifically:

1. **Detect:** Does delaying exposure to a specific skill (arithmetic, a syntax construction) during pretraining cause a permanent performance cap — even when the model eventually receives the *same total amount* of that skill's data, just later?
2. **Reopen:** If so, can you deliberately undo it with a targeted intervention (embedding reset, LR spike, layer freezing, concentrated re-exposure)?
3. **Diagnose:** When recovery happens, did the model repair its original internal circuit for that skill, or build an entirely new one to route around the damage?

The hook: this mirrors amblyopia treatment in children — early-enough intervention can partially reverse vision loss from a missed critical window. Nobody has tested whether an analogous "treatment" works for a missed skill window in LLM pretraining, or looked inside the network to see *how* recovery happens.

WHY THIS IS A STRONG, NOVEL PAPER

- A plain "critical periods exist in LLMs" finding is expected at this point — reviewers have seen versions of this claim before, and one existing paper (Constantinescu et al. 2024, see below) even found a *null* result for a related setup.
- "Here's whether and how you can undo it" is actionable and gives you a clean, intuitive framing that (unlike a lot of ML papers) doesn't need a long intro to justify why anyone should care.
- A dissociation result — e.g. "arithmetic is reopenable via embedding reset, syntax isn't" — is exactly the kind of clean, surprising finding reviewers respond to.
- Your circuit-diagnosis layer (same circuit vs. new circuit) is not tested by any existing paper in this space. That alone is a real contribution even if your intervention results are mixed.

THE 5 KEY PAPERS (read in this order)

1. Achille, Rovere & Soatto (2019) — "Critical Learning Periods in Deep Neural Networks" https://arxiv.org/pdf/1711.08856 The foundational paper. A temporary deficit (blurred images) early in a vision network's training causes permanent damage, mirroring biological critical periods; the same deficit late in training is easily overcome. They use Fisher Information to show why: information rises fast early in training then decreases, preventing later redistribution of information resources. Deeper networks show it more strongly.

2. Kleinman, Achille & Soatto (2023 & 2024) — two related papers

- Multisensory integration (CVPR 2023): https://arxiv.org/pdf/2210.04643 — shows integrating multiple correlated signals depends on early exposure; interference early on permanently impairs the resulting skill, driven by unstable early training dynamics.
- Deep linear networks (2024): https://arxiv.org/pdf/2308.12221 — shows critical periods emerge even in the simplest possible (linear) networks, suggesting this is a fundamental property of learning itself, not an architecture-specific quirk.

3. Constantinescu, Pimentel, Cotterell & Warstadt (2024) — "Do Language Models Have a Critical Period for Language Acquisition?" https://arxiv.org/html/2407.19325v2 Directly relevant prior work. Tested sequential L1→L2 language exposure in LMs at varying delays. Finding: LMs do **not** naturally show critical period effects for delayed second-language exposure — human-like effects only appeared when they artificially reduced the model's plasticity using Elastic Weight Consolidation (EWC), a regularizer. This is the paper whose null result your research directly engages with — but note their setup (whole new language, post-hoc) is meaningfully different from yours (specific skill, within one language's pretraining).

4. Chen et al. (2023) — "Improving Language Plasticity via Pretraining with Active Forgetting" https://ar5iv.labs.arxiv.org/html/2307.01163 Basis for your embedding-reset intervention. They periodically reset the embedding layer *throughout* pretraining as a standing policy (not a rescue), and tested it on adapting an already-pretrained model to an entirely new language. Result: faster adaptation, better low-data performance, especially for languages distant from English. Your research differs in three ways: (a) you apply it retroactively to fix a specific missed skill, not as a built-in policy, (b) you test a skill within one language, not a whole new language, (c) you compare it against 3 alternative interventions and add circuit-level analysis — none of which exists in this paper.

5. "Curriculum Learning for LLM Pretraining: An Analysis of Learning Dynamics" (2026) https://arxiv.org/html/2601.21698 Closest to your syntax/timing angle. Trained models 14M-1B parameters under different data-ordering curricula. Training follows shared latent phases; curricula mainly change time spent per phase. Critically: random ordering produces noisier gradients and worse final accuracy specifically in smaller models (14M-160M params) — direct evidence that ordering has real, measurable effects at exactly

your model scale, without any artificial plasticity reduction. This creates a genuine tension with paper #3's null result — your study is positioned to help explain the discrepancy.

CRITICAL PITFALLS — KEEP THESE VISIBLE THROUGHOUT THE ENTIRE STUDY

Pitfall	What It Means	How to Fix It
Catastrophic forgetting vs. critical period	An intervention might "recover" the delayed skill by overwriting other capabilities — the model gets good at arithmetic again but gets worse at general English. That's not real reopening, it's a different tradeoff.	Every reopening evaluation must track **two things simultaneously**: (1) recovery of the delayed skill, (2) retention of general language capability (held-out perplexity), measured before and after each intervention. A "recovery" that tanks general capability should be flagged as forgetting, not reopening.
Pre-training contamination	If you start from a standard open-source checkpoint (GPT-2, Pythia, etc.), that model has already passed its critical period on standard web data — you can't control when it saw arithmetic or syntax.	Train every model **entirely from scratch**, from random initialization, using your own controlled data mixture where you dictate exactly when each skill enters the training stream. No pretrained checkpoints anywhere, including baselines.

FINALIZED SPECS (locked in — don't re-decide these)

Skills to test:

- **Arithmetic:** 2-4 digit addition; carrying vs. no-carrying as a difficulty axis. Synthetic, infinite supply, perfectly gradeable.
- **Syntax:** long-distance wh-question extraction (object-gap constructions) — chosen because paper #5 already found ordering effects here, giving you a direct comparison point and existing BLiMP test suites.
- **Negative control:** word-frequency/vocabulary prediction — should NOT show a critical period; if it does, that flags a problem with your method, not your hypothesis.

Dataset: BabyLM Challenge "strict-small" corpus (~10M words) as the natural-language base — purpose-built for small-model, developmental-scale training, and gives you a citable, established baseline. Synthetic arithmetic layered on top, injected on your own delay schedule.

Model architecture: 10M-30M parameters, 4-6 layers, hidden dim 256-512, 8 attention heads, sequence length 256-512 tokens. Plenty of capacity for these skills, well within your RTX 4060's 8GB VRAM.

Engineering setup (RTX 4060 / Ryzen 7 7840 optimized):

- Mixed precision (bfloat16 via `torch.autocast`) — halves memory use, leverages Ada Lovelace tensor cores
- FlashAttention-2 — linear rather than quadratic memory scaling for attention
- Gradient accumulation — micro-batches of 4-8 accumulated before `optimizer.step()`
- `torch.compile()` — free ~10-20% speedup
- Pre-tokenize the dataset into a `numpy.memmap` binary file; `DataLoader` with `num_workers=8` to keep the GPU fed
- Clean OOP structure: subclass `nn.Module` per attention/FFN block so components can be isolated and reset dynamically mid-training
- Training loop accepts external triggers — manual LR overrides or `requires_grad=False` toggling fired at exact checkpoints
- Log checkpoint evals directly into pandas DataFrames during training; `scipy.stats` for significance testing; `seaborn` for comparative loss curves and heatmaps

---

YOUR HYPOTHESIS DOCUMENT

Operational definitions:

- **Critical Period (Detect):** Skill S has a critical period ending at checkpoint T if a model first exposed to S after T reaches significantly lower final accuracy than a model exposed from the start — even with matched total S-token exposure — and the gap persists after extended fine-tuning on S alone.
- **Reopening:** An intervention successfully reopens S's critical period if it brings the delayed model's accuracy to within a set margin (e.g. 90%) of the early-exposure baseline, *without* degrading general-language perplexity beyond a matched threshold. Recovery at the cost of general capability counts as forgetting, not reopening.
- **Circuit Identity (Diagnose):** A recovered model *repaired the original circuit* if activation patching at the baseline's relevant layers/heads reproduces its skill behavior, and ablating those components damages performance similarly to the baseline. It *built a new circuit* if patching/ablating the baseline's components has little effect on the recovered model.

Predicted results:

1. Which skill shows a stronger critical period — genuinely open. Arithmetic's discrete, compositional structure could make it either easier (few correct patterns to relearn) or

harder (little partial-credit gradient signal) to reopen than syntax's distributed, long-range dependency structure.

2. Most effective intervention — **embedding/layer reset**, following Chen et al.'s precedent that wiping over-committed representations restores plasticity. Predicted ordering: reset > cram > LR spike > layer freeze (worth testing, not assuming).
3. Circuit identity of recovered models — genuinely open, but linked to prediction #2: if reset wins, expect "new circuit" more often, since reset destroys the original representation entirely; gentler interventions (LR spike, freeze) that preserve existing weights are more likely to show "repaired circuit" instead.

Every one of these can come out wrong and still be publishable — that's the point of writing predictions down before running anything.

WEEK-BY-WEEK WALKTHROUGH

Week 1 — Literature + Hypothesis

- Read the 5 papers above (a few hours each is enough — understand method + finding, no need to reproduce)
- Confirm you're comfortable with the finalized skills/dataset/architecture above
- Finalize your predicted-results doc (above) before running any code
- Can run in parallel with Week 2 infrastructure work to save time

Week 2 — Infrastructure

- Build a nanoGPT-style training pipeline, random-initialization-only (see brief below for the exact handoff spec)
- Build data generators: synthetic arithmetic, syntax-filtered corpus from BabyLM, delay scheduler
- Build the intervention module: swappable functions for reset/LR-spike/freeze/cram
- Build the eval harness logging **both** skill accuracy and general perplexity at every checkpoint from day one
- Sanity-check with one full baseline run

Week 3 — Pilot: Detect

- Small grid: 1 model size × 3 delay onsets × 2 skills
- Confirm total-exposure is matched across conditions (only timing differs)
- Check for any delay-dependent performance gap — this validates your method before scaling up

Week 4 — Pilot: Reopen

- Take delayed-exposure models from Week 3 that show a gap
- Apply each intervention type at a fixed point, continue training with equal budget
- Check for recovery on the target skill **and** confirm general perplexity didn't degrade — flag forgetting now, not later
- Validates the reopening mechanism works at all before scaling up

Weeks 5-7 — Full Experiment Suite

- Detect: 3 model sizes × 4-5 delays × 2-3 skills × 3 seeds
- Reopen: for skill/delay combos showing a real gap, run all 4 intervention types × 3 seeds
- Small models on your GPU finish in hours — batch overnight across these 3 weeks
- Checkpoint review after week 5-6 to catch issues early

Week 8 — Circuit Diagnosis

Don't skip this even under time pressure — it's the mechanistic core of the paper.

- **Activation patching / causal tracing:** does patching the recovered model's activations at the baseline's relevant layers/heads reproduce its behavior? If yes → likely reused pathway; if no → likely rerouted.
- **Weight/representation similarity:** compare relevant weights/hidden representations (CKA or cosine similarity) between recovered model and baseline on the skill task.
- **Ablation transfer test:** ablate the components the baseline relies on for the skill — does it break the recovered model equally, less, or not at all? "Not at all" → strong evidence of a new circuit.
- **Pruning-based localization:** find the minimal subnetwork sufficient for the skill in both models, compare footprint overlap.
- Bonus: does "same circuit vs. new circuit" depend on which intervention was used?

Week 9 — Statistical Analysis

- Detect: mixed-effects regression, final accuracy ~ delay × model_size, with confidence intervals
- Reopen: recovery percentage ~ intervention_type × skill × delay_onset
- Diagnose: circuit-overlap metrics ~ intervention_type × skill, reported alongside recovery percentages
- Compare arithmetic vs. syntax reopenability, and whether model size changes which interventions work or which circuit outcome results

Weeks 10-11 — Writing

- Structure: Abstract, Intro (amblyopia-treatment hook), Related Work (the 5 papers), Method (Detect + Reopen + Diagnose), Results, Discussion/Limitations, Conclusion
- Be upfront about small-model limitations and about interventions that failed, not just ones that worked
- Direct engagement with Constantinescu et al.'s null result — your skill-specific + intervention + circuit framing is the differentiator

Weeks 12-13 — Polish & Submit

- Feedback pass, code cleanup + public release
- Target venues: BabyLM workshop (strong fit given developmental framing), ACL/ICLR/NeurIPS workshops, or arXiv + main conference submission if results are strong

---

THE 4 REOPENING METHODS (Tier 1 — try these first)

1. **Embedding/layer reset** — reset embedding (or specific layers) to fresh random init partway through training, then continue. Based on Chen et al.'s active-forgetting precedent. Biggest forgetting risk, so monitor general perplexity closely here.
2. **Learning rate spike** — temporarily boost LR right before/during re-exposure, then decay back down. Gentlest option — good baseline for "does anything happen at all."
3. **Layer freeze/unfreeze** — freeze most of the network, unfreeze only specific layers during re-exposure. Tests whether recovery is localized or needs whole-network updates.
4. **Concentrated re-exposure ("cram phase")** — feed a burst of unusually high-density skill data for a short stretch. This is your control: if cram alone fully recovers performance, that suggests the issue was exposure quantity, not plasticity — undercutting the critical-period story. The other three are your actual test of whether plasticity needs restoring.

TIER 2 FALLBACK METHODS (only if all 4 above fail)

If Tier 1 shows zero recovery across all skills, run Tier 2 on just your best-candidate skill/delay combo rather than the whole grid (bounds the added time to ~1-2 weeks instead of doubling the study):

- **Optimizer state reset** — reset Adam's running gradient-moment estimates without touching weights; gentler than a full LR spike
- **Cyclical LR restarts** — repeated cosine-annealing restarts instead of one spike, giving multiple chances to escape the current basin

- **Adapter/LoRA modules** — bolt on a small trainable module dedicated to the missed skill instead of forcing the frozen network to relearn it. If this is what works, that itself is a finding: the skill isn't reachable in the original circuit at all.
- **Net2Net-style network growth** — literally expand the network at the intervention point with fresh parameters
- **Lottery-ticket rewind** — rewind a subnetwork's weights to an earlier checkpoint (before the critical period closed) and retrain just that subnetwork
- **Spaced repetition** — interleave the missed skill at increasing/varying intervals, rather than one dense cram burst
- **Replay of early data + skill** — reintroduce a slice of the original early corpus alongside the new skill, in case the issue is distributional shift rather than pure plasticity loss
- **Distillation from a "healthy" twin** — train a same-size model that got the skill early, then distill its behavior into the delayed model, testing whether the delayed model can absorb the right circuit if guided

---

THE CURRICULUM-DESIGN ANGLE (for your paper's motivation section)

Real multi-skill LLMs aren't built by training separate specialist models and merging them — model merging (weight averaging) works for combining independently fine-tuned variants of the *same* base model on non-dependent tasks, but doesn't reliably compose hierarchical knowledge (e.g. a finance-reasoning model that genuinely depends on the model having internalized arithmetic first). Real practice is one model trained on a well-designed mixture/curriculum of all target domains at once. Your finding — if delaying a skill past checkpoint X causes permanent capability loss — is directly useful for anyone designing that curriculum's ordering. This is a legitimate motivating/applications point for your paper, not a new experiment to run.

FABLE 5 / CODING HANDOFF BRIEF

Use this as a self-contained brief to hand to your coding assistant to kick off Week 2 without re-explaining context.

Build first (this week):

1. Basic nanoGPT-style model + training loop, random-init only, running on the BabyLM corpus alone (sanity check — confirm loss decreases sensibly)
2. Synthetic arithmetic data generator + syntax-filtered data extraction from BabyLM/BLiMP
3. Delay scheduler wired into the data pipeline
4. Eval harness logging both skill accuracy and general perplexity at every checkpoint

Hold off on for now:

- Intervention module (reset/LR-spike/freeze/cram) — not needed until Week 3's pilot confirms a critical period exists
- Circuit diagnosis tooling (activation patching, ablation, pruning) — needed only once recovery is observed, around Week 8

Full engineering spec: model size (10M-30M params, 4-6 layers, hidden dim 256-512, 8 heads, seq len 256-512), bfloat16 mixed precision, FlashAttention-2, gradient accumulation (micro-batch 4-8), `torch.compile()`, memmap pre-tokenization, `num_workers=8` DataLoader, OOP module structure for dynamic resets, external-trigger-capable training loop, pandas/scipy/seaborn for logging and analysis.

WHAT WOULD MAKE THIS A HIT

1. A clean dissociation: some skill/intervention combos recover, others don't
2. A mechanistic explanation tied to circuit-level evidence, not just accuracy numbers
3. Direct, honest engagement with Constantinescu et al.'s null result — explaining why your framing gives a different answer
4. Reproducible, publicly released code

IMMEDIATE NEXT STEP

Start Week 1 today: read the 5 papers, confirm the hypothesis document, and hand the Fable 5 brief to your coding assistant to start Week 2's infrastructure in parallel.